EXPERIMENT 15

Write the Python program to implement a Decision Tree

Register Number: **192511013**

192511013
192511013
192511013

Aim

To write a Python program to build a decision tree classifier from a training data set (Play Tennis) using the ID3 algorithm, based on entropy and information gain, and to use the tree to classify a new sample.

Algorithm

Step 1: Store the training data: each row has the attribute values (Outlook, Temperature, Humidity, Wind) and the class label (Play = Yes / No).

Step 2: Calculate the entropy of a set S as Entropy(S) = - sum of p(i) x log2 p(i), where p(i) is the proportion of rows in class i.

Step 3: For every attribute A, calculate Gain(S, A) = Entropy(S) - sum of (|Sv| / |S|) x Entropy(Sv), where Sv is the subset of rows having value v for A.

Step 4: Select the attribute with the highest information gain as the decision node (root).

Step 5: Create a branch for every value of that attribute and divide the rows accordingly.

Step 6: For each branch: if all rows have the same class, create a leaf with that class; otherwise repeat steps 3 to 5 on the subset using the remaining attributes. If no attributes remain, use the majority class.

Step 7: Print the entropy, information gains and the final tree. To classify a new sample, follow the branches matching its attribute values from the root until a leaf is reached.

192511013

Program

```
import math
from collections import Counter

# Play-Tennis data set: (Outlook, Temperature, Humidity, Wind) -> Play
features = ["Outlook", "Temperature", "Humidity", "Wind"]
data = [
    ("Sunny", "Hot", "High", "Weak", "No"),
    ("Sunny", "Hot", "High", "Strong", "No"),
    ("Overcast", "Hot", "High", "Weak", "Yes"),
    ("Rain", "Mild", "High", "Weak", "Yes"),
    ("Rain", "Cool", "Normal", "Weak", "Yes"),
    ("Rain", "Cool", "Normal", "Strong", "No"),
    ("Overcast", "Cool", "Normal", "Strong", "Yes"),
    ("Sunny", "Mild", "High", "Weak", "No"),
    ("Sunny", "Cool", "Normal", "Weak", "Yes"),
    ("Rain", "Mild", "Normal", "Weak", "Yes"),
    ("Sunny", "Mild", "Normal", "Strong", "Yes"),
    ("Overcast", "Mild", "High", "Strong", "Yes"),
    ("Overcast", "Hot", "Normal", "Weak", "Yes"),
    ("Rain", "Mild", "High", "Strong", "No"),
]

def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, col):
    total = len(rows)
    remainder = 0
    for value in set(r[col] for r in rows):
        subset = [r for r in rows if r[col] == value]
        remainder += len(subset) / total * entropy(subset)
    return entropy(rows) - remainder

def build_tree(rows, available):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not available:
        return Counter(labels).most_common(1)[0][0]
    best = max(available, key=lambda c: info_gain(rows, c))
    tree = {features[best]: {}}
    for value in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == value]
        rest = [c for c in available if c != best]
        tree[features[best]][value] = build_tree(subset, rest)
    return tree

def print_tree(tree, indent=0):
    if not isinstance(tree, dict):
        print(' ' * indent + '-> ' + tree)
        return
    attr = next(iter(tree))
    for value, sub in tree[attr].items():
        print(' ' * indent + f"[{attr} = {value}]")
        print_tree(sub, indent + 4)

def predict(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][sample[attr]]
    return tree

if __name__ == "__main__":
    print(f"Entropy of the whole data set: {entropy(data):.3f}\n")
    print("Information gain of each attribute:")
```

```
for i, name in enumerate(features):
    print(f"  {name:<12}: {info_gain(data, i):.3f}")
tree = build_tree(data, list(range(len(features))))
print("\nDecision tree (ID3):")
print_tree(tree)
sample = {"Outlook": "Sunny", "Temperature": "Cool", "Humidity": "High", "Wind": "Strong"}
print("\nNew sample:", sample)
print("Prediction (Play Tennis?):", predict(tree, sample))
```

192511013

Output

```
Entropy of the whole data set: 0.940

Information gain of each attribute:
  Outlook     : 0.247
  Temperature : 0.029
  Humidity    : 0.152
  Wind        : 0.048

Decision tree (ID3):
[Outlook = Overcast]
    -> Yes
[Outlook = Rain]
    [Wind = Strong]
        -> No
    [Wind = Weak]
        -> Yes
[Outlook = Sunny]
    [Humidity = High]
        -> No
    [Humidity = Normal]
        -> Yes

New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
Prediction (Play Tennis?): No
```

192511013